

Hard Link vs. Soft Link

Part A — Create the Links

The terminal screenshot shows creation of **link_lab**, creation of **security.txt**, creation of a hard link named **security_hard.txt**, and a symbolic link named **security_soft.txt**. The inode listing confirms that the original and hard link share the same inode, while the symbolic link has a different inode.

Screenshot 1 — Part A: File creation and inode listing

```
srinivas@DESKTOP-E4HFS1K:~$ mkdir link_lab
srinivas@DESKTOP-E4HFS1K:~$ cd link_lab
srinivas@DESKTOP-E4HFS1K:~/link_lab$ printf "Linux File System Security Lab\nConfidential Security Report\n" > security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ cat security.txt
Linux File System Security Lab
Confidential Security Report
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ln security.txt security_hard.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ln -s security.txt security_soft.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ls -li security.txt security_hard.txt security_soft.txt
15762598695972854 -rw-rw-r-- 2 srinivas srinivas 60 Sep  3 11:50 security.txt
15762598695972854 -rw-rw-r-- 2 srinivas srinivas 60 Sep  3 11:50 security_hard.txt
37154696925975271 lrwxrwxrwx 1 srinivas srinivas 12 Sep  3 11:50 security_soft.txt -> security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$
```

Actual terminal evidence supplied by the student.

File	Observed inode number
security.txt	15762598695972854
security_hard.txt	15762598695972854
security_soft.txt	37154696925975271

What difference do you observe?

Hard Link: Shares the **same inode**, increases the file's **link count**, and mirrors the exact file size.

Soft Link: Gets a **new inode**, starts with an **l** (link) attribute, and its size matches only the length of the filename text it points to.

Part B — Examine File Metadata

The student's **ls -l** and **stat** output shows both regular files with permissions **-rw-rw-r--**, while the symbolic link begins with **l** and has permissions displayed as **lrwxrwxrwx**.

Screenshot 2 — Part B: ls -l and stat

```
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ls -l security.txt security_hard.txt security_soft.txt
-rw-rw-r-- 2 srinivas srinivas 60 Sep  3 11:50 security.txt
-rw-rw-r-- 2 srinivas srinivas 60 Sep  3 11:50 security_hard.txt
lrwxrwxrwx 1 srinivas srinivas 12 Sep  3 11:50 security_soft.txt -> security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ stat security.txt security_hard.txt security_soft.txt
  File: security.txt
  Size: 60          Blocks: 0          IO Block: 4096   regular file
Device: 0,2      Inode: 15762598695972854  Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/srinivas)   Gid: ( 1000/srinivas)
Access: 2026-09-03 11:50:25.155443700 +0530
Modify: 2026-09-03 11:50:17.235158400 +0530
Change: 2026-09-03 11:50:35.360141400 +0530
Birth: -
  File: security_hard.txt
  Size: 60          Blocks: 0          IO Block: 4096   regular file
Device: 0,2      Inode: 15762598695972854  Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/srinivas)   Gid: ( 1000/srinivas)
Access: 2026-09-03 11:50:25.155443700 +0530
Modify: 2026-09-03 11:50:17.235158400 +0530
Change: 2026-09-03 11:50:35.360141400 +0530
Birth: -
  File: security_soft.txt -> security.txt
  Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 0,2      Inode: 37154696925975271  Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/srinivas)   Gid: ( 1000/srinivas)
Access: 2026-09-03 11:50:42.790950600 +0530
Modify: 2026-09-03 11:50:42.790950600 +0530
Change: 2026-09-03 11:50:42.790950600 +0530
Birth: -
srinivas@DESKTOP-E4HFS1K:~/link_lab$
```

Actual metadata output from the student's terminal.

Parameter	Original	Hard Link	Soft Link
Inode number	15762598695972854	15762598695972854	37154696925975271
File type	Regular file	Regular file	Symbolic link
Size	60 bytes	60 bytes	12 bytes
Link count	2	2	1
Permissions	-rw-rw-r--	-rw-rw-r--	lrwxrwxrwx

Explanation: The link count is 2 for the original and hard link because two directory entries reference the same inode. The symbolic link has its own inode and therefore its own link count of 1. The symbolic link is 12 bytes because its stored target pathname is **security.txt**.

Part C — Investigate readlink:

The screenshot shows that **readlink security_soft.txt** displays **security.txt**. Running readlink on the hard link produces no visible target pathname. The **-f** option resolves the symbolic link to its canonical absolute path.

Screenshot 3 — Part C: readlink

```
srinivas@DESKTOP-E4HFS1K:~/link_lab$ readlink security_soft.txt
security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ readlink security_hard.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ readlink -f security_soft.txt
/home/srinivas/link_lab/security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ _
```

The **-f** command resolves to **/home/srinivas/link_lab/security.txt**.

Answers

readlink security_soft.txt: displays the pathname stored in the symbolic link: **security.txt**.

readlink security_hard.txt: no target pathname is displayed because a hard link does not store a pathname to another file.

readlink -f: reveals the canonical absolute target path: **/home/srinivas/link_lab/security.txt**.

Part D — Content Modification

The screenshot demonstrates appending **Unauthorized login detected** through security.txt, reading the content through all three names, then appending **Investigation initiated** through the hard link. Both the original name and the symbolic link show the modification.

Screenshot 4 — Part D: Content modification

```
srinivas@DESKTOP-E4HFS1K:~/link_lab$ echo "Unauthorized login detected" >> security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ cat security.txt
Unauthorized login detected
srinivas@DESKTOP-E4HFS1K:~/link_lab$ cat security_hard.txt
Unauthorized login detected
srinivas@DESKTOP-E4HFS1K:~/link_lab$ cat security_soft.txt
Unauthorized login detected
srinivas@DESKTOP-E4HFS1K:~/link_lab$ echo "Investigation initiated" >> security_hard.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ cat security.txt
Unauthorized login detected
Investigation initiated
srinivas@DESKTOP-E4HFS1K:~/link_lab$ cat security_soft.txt
Unauthorized login detected
Investigation initiated
srinivas@DESKTOP-E4HFS1K:~/link_lab$ _
```

Changes made through either filename are visible through the other links.

Observation

The newly added content is visible through both links because the hard link and original refer to the same inode/data. The symbolic link follows the target pathname and therefore also reads the updated target file. Modifying the file through security_hard.txt changes the same underlying data.

Part E — Rename Experiment & Part F — Deletion Experiment Screenshot 5 — Rename and deletion experiments

```
srinivas@DESKTOP-E4HFS1K:~/link_lab$ printf "Linux File System Security Lab\nConfidential Security Report\n" > security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ln security.txt security_hard.txt
ln: failed to create hard link 'security_hard.txt': File exists
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ln -s security.txt security_soft.txt
ln: failed to create symbolic link 'security_soft.txt': File exists
srinivas@DESKTOP-E4HFS1K:~/link_lab$ rm security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ cat security_hard.txt
Unauthorized login detected
Investigation initiated
srinivas@DESKTOP-E4HFS1K:~/link_lab$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ls -l security_hard.txt security_soft.txt
-rw-rw-r-- 2 srinivas srinivas 52 Sep  3 12:04 security_hard.txt
lrwxrwxrwx 1 srinivas srinivas 12 Sep  3 11:50 security_soft.txt -> security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ readlink security_soft.txt
security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ security_soft.txt -> security.txt
security_soft.txt: command not found
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ls -li security_hard.txt
15762598695972854 -rw-rw-r-- 2 srinivas srinivas 52 Sep  3 12:04 security_hard.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ _
```

Rename result: security_hard.txt continues to work after security.txt is renamed to incident.txt. security_soft.txt fails because it still stores the pathname **security.txt**.

Deletion result: after deleting security.txt, security_hard.txt remains accessible because the hard link still references the inode. security_soft.txt becomes a dangling/broken symbolic link.

Link-count justification: deleting one hard-link name reduces the inode link count from 2 to 1; the underlying data remains while at least one hard link still exists.

Part G — Directory and Filesystem Behaviour Screenshot 6 — Part G:

Directory linking

```
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ^C
srinivas@DESKTOP-E4HFS1K:~/link_lab$ mkdir evidence
mkdir: cannot create directory 'evidence': File exists
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ln evidence evidence_hard
ln: evidence: hard link not allowed for directory
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ls -l
total 0
drwxrwxr-x 1 srinivas srinivas 4096 Sep  3 12:33 evidence
lrwxrwxrwx 1 srinivas srinivas  8 Sep  3 12:33 evidence_link -> evidence
-rw-rw-r-- 2 srinivas srinivas  52 Sep  3 12:04 incident.txt
-rw-rw-r-- 1 srinivas srinivas  0 Sep  3 12:30 security.txt
-rw-rw-r-- 2 srinivas srinivas  52 Sep  3 12:04 security_hard.txtlrwxrwxrwx 1 srinivas srinivas  12 Sep  3 11:50 security_soft.txt -> security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ln -s evidence evidence_link
srinivas@DESKTOP-E4HFS1K:~/link_lab$ ls -l
total 0
drwxrwxr-x 1 srinivas srinivas 4096 Sep  5 10:23 evidence
lrwxrwxrwx 1 srinivas srinivas  8 Sep  3 12:33 evidence_link -> evidence
-rw-rw-r-- 2 srinivas srinivas  52 Sep  3 12:04 incident.txt
-rw-rw-r-- 1 srinivas srinivas  0 Sep  3 12:30 security.txt
-rw-rw-r-- 2 srinivas srinivas  52 Sep  3 12:04 security_hard.txtlrwxrwxrwx 1 srinivas srinivas  12 Sep  3 11:50 security_soft.txt -> security.txt
srinivas@DESKTOP-E4HFS1K:~/link_lab$ _
```

Hard link to directory: Failed with the message **hard link not allowed for directory**. **Symbolic link to directory:** Successful; **evidence_link -> evidence** is shown by `ls -l`. **Directory comparison:** hard links cannot normally target directories, while symbolic links can.

Cross-filesystem note: the provided screenshots do not contain a successful experiment using two actual different filesystems/mount points. Therefore, no additional cross-filesystem result is claimed from the screenshots.

Final Comparison

Parameter	Hard Link	Symbolic Link
Inode number	Same as target	Different / independent inode
File type	Regular file (-)	Symbolic link (l)
Link count	Target link count increases	Separate link inode, normally 1
File size	Same as target	Length of stored pathname
Stores target pathname?	No	Yes
readlink output	No target pathname	Displays target pathname
Modifying target	Changes visible	Changes visible
Renaming target	Continues working	Usually breaks
Deleting target	Works if another hard link exists	Becomes dangling/broken
Can become dangling?	No	Yes
Directory linking	Not allowed normally	Allowed
Cross-filesystem linking	Not supported	Supported in principle

SORT — Questions 1–6

Screenshot 7 — SORT Questions 1–6

```
srinivas@DESKTOP-E4HFS1K:~/link_lab$ #1
srinivas@DESKTOP-E4HFS1K:~/link_lab$ sort -k3 -n alerts.txt
172.16.0.5 guest 2
10.0.0.20 admin 8
10.0.0.8 root 12
srinivas@DESKTOP-E4HFS1K:~/link_lab$ #2
srinivas@DESKTOP-E4HFS1K:~/link_lab$ awk '{print $1}' alerts.txt | sort
10.0.0.20
10.0.0.8
172.16.0.5
srinivas@DESKTOP-E4HFS1K:~/link_lab$ #3
srinivas@DESKTOP-E4HFS1K:~/link_lab$ awk '{print $1}' alerts.txt | sort -u
10.0.0.20
10.0.0.8
172.16.0.5
srinivas@DESKTOP-E4HFS1K:~/link_lab$ #4
srinivas@DESKTOP-E4HFS1K:~/link_lab$ awk '{print $1}' alerts.txt | sort -n
10.0.0.20
10.0.0.8
172.16.0.5
srinivas@DESKTOP-E4HFS1K:~/link_lab$ awk '{print $3}' alerts.txt | sort
-n
2
8
12
srinivas@DESKTOP-E4HFS1K:~/link_lab$ #5
srinivas@DESKTOP-E4HFS1K:~/link_lab$ awk '{print $3}' alerts.txt | sort -nr
12
8
2
srinivas@DESKTOP-E4HFS1K:~/link_lab$ #6
srinivas@DESKTOP-E4HFS1K:~/link_lab$ awk '{print $2}' alerts.txt | sort
-f
admin
guest
root
```

Actual terminal evidence supplied by the student.

Q	Command used	Purpose
1	sort -k3 -n alerts.txt	Sort records by failed attempts
2	awk '{print \$1}' alerts.txt sort	Sort IP addresses alphabetically
3	awk '{print \$1}' alerts.txt sort -u	Display unique IP addresses
4	awk '{print \$3}' alerts.txt sort -n	Sort numbers numerically
5	awk '{print \$3}' alerts.txt sort -nr	Sort numbers descending
6	awk '{print \$2}' alerts.txt sort -f	Sort usernames ignoring case

SORT — Questions 7–11

Screenshot 8 — SORT Questions 7–11

```
srinivas@DESKTOP-E4HFS1K:~/link_lab$ #7
srinivas@DESKTOP-E4HFS1K:~/link_lab$ awk '{print $1}' alerts.txt | sort
| uniq -c | sort -nr
  1 172.16.0.5
  1 10.0.0.8
  1 10.0.0.20
srinivas@DESKTOP-E4HFS1K:~/link_lab$ #8
srinivas@DESKTOP-E4HFS1K:~/link_lab$ sort -k3 -n alerts.txt
172.16.0.5 guest 2
10.0.0.20 admin 8
10.0.0.8 root 12
srinivas@DESKTOP-E4HFS1K:~/link_lab$ #9
srinivas@DESKTOP-E4HFS1K:~/link_lab$ sort -k3 -n alerts.txt
172.16.0.5 guest 2
10.0.0.20 admin 8
10.0.0.8 root 12
srinivas@DESKTOP-E4HFS1K:~/link_lab$ #10
srinivas@DESKTOP-E4HFS1K:~/link_lab$ grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' alerts.txt | sort | uniq
10.0.0.20
10.0.0.8
172.16.0.5
srinivas@DESKTOP-E4HFS1K:~/link_lab$ #11
srinivas@DESKTOP-E4HFS1K:~/link_lab$ sort -k3 -nr alerts.txt | awk '{print $1}' | head -n 5
10.0.0.8
10.0.0.20
172.16.0.5
srinivas@DESKTOP-E4HFS1K:~/link_lab$ _
```

Actual terminal evidence supplied by the student.

Q	Command used	Purpose
7	awk '{print \$1}' alerts.txt sort uniq -c sort -nr	Count IP occurrences; most frequent first
8	sort -k3 -n alerts.txt	Sort records by failed attempts
9	sort -k3 -n alerts.txt	Sort the supplied lab records by score/attempt
10	grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' alerts.txt sort uniq	Extract and uniquely sort IPs
11	sort -k3 -nr alerts.txt awk '{print \$1}' head -n 5	Display top 5 IP addresses

Note on Question 7: the screenshot shows one occurrence for each of the three visible IPs. The exact displayed count depends on the current contents of alerts.txt.

Note on Question 11: the provided screenshot shows the four IP records ordered by the failed-attempt field; because only four data records are present, fewer than five IP addresses can appear.

Conclusion

The experiments demonstrate the central difference between hard and symbolic links: a hard link is another name for the same inode, while a symbolic link is a separate file containing a pathname to its target. The terminal evidence shows that modifications are visible through all valid names, while renaming or deleting the target pathname can break a symbolic link but does not remove the underlying file when a hard link remains.

The sort exercises demonstrate field-based numeric sorting, reverse sorting, case-insensitive sorting, deduplication with sort/uniq, IP extraction with grep, and a pipeline for ranking IP addresses.